

Deep Learning & Neural Networks

Oct. 28, 2025 - Lecture 7

Outline

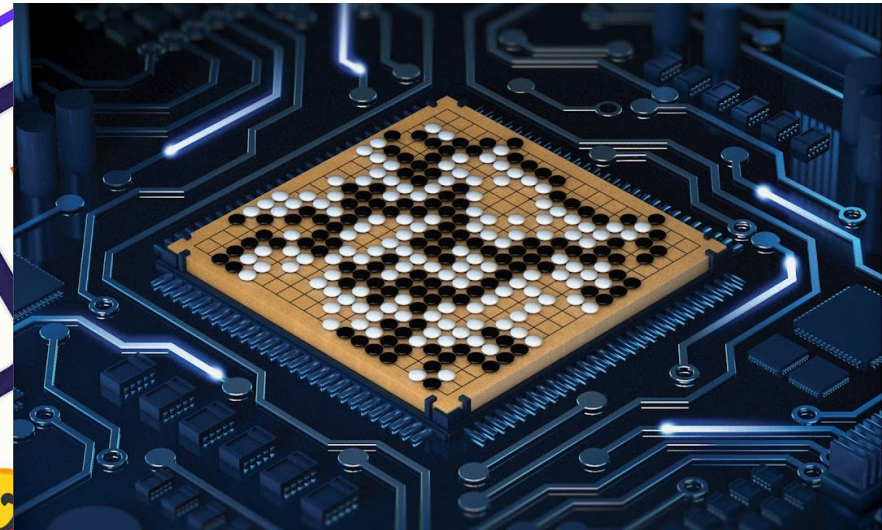
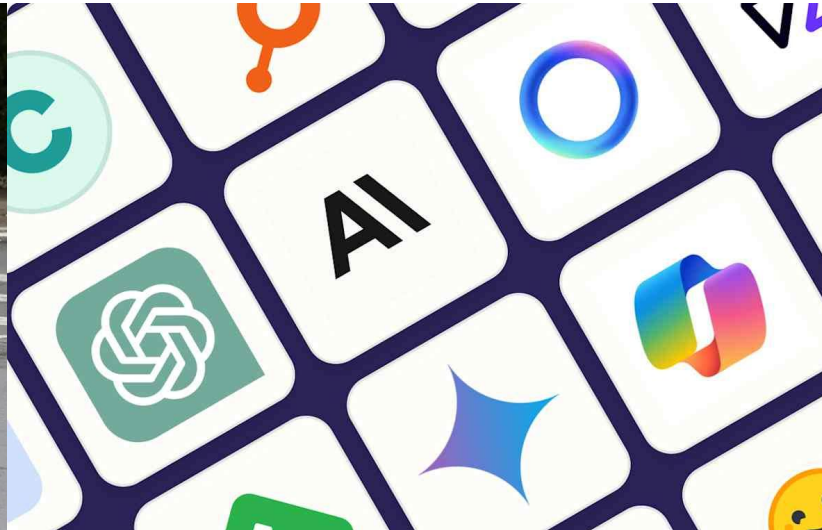
- Neural networks background, NN basics
- Activation functions
- Single layer NNs (Perceptron)
- Multi-layer NNs (MLPs)
- Training NNs
- Applications of Neural Networks
- Challenges with Neural Networks

Recap

- So far, we've covered some **linear models** (linear and logistic regression) and a few **non-linear models** (trees, SVM) for relatively simple prediction tasks.
- What about problems such as image or speech recognition?
 - These kinds of problems are trivial for humans.
 - The models that we've covered so far are **not complex enough** to perform well at these tasks.

Applications of NNs

- **Vision:** Image classification (e.g., recognizing cats/dogs)
- **Speech:** Speech-to-text, speech-recognition, auto-captions
- **NLP:** Chatbots, translation
- **Games:** AlphaGo, reinforcement learning



Neural Networks: Biological Inspiration

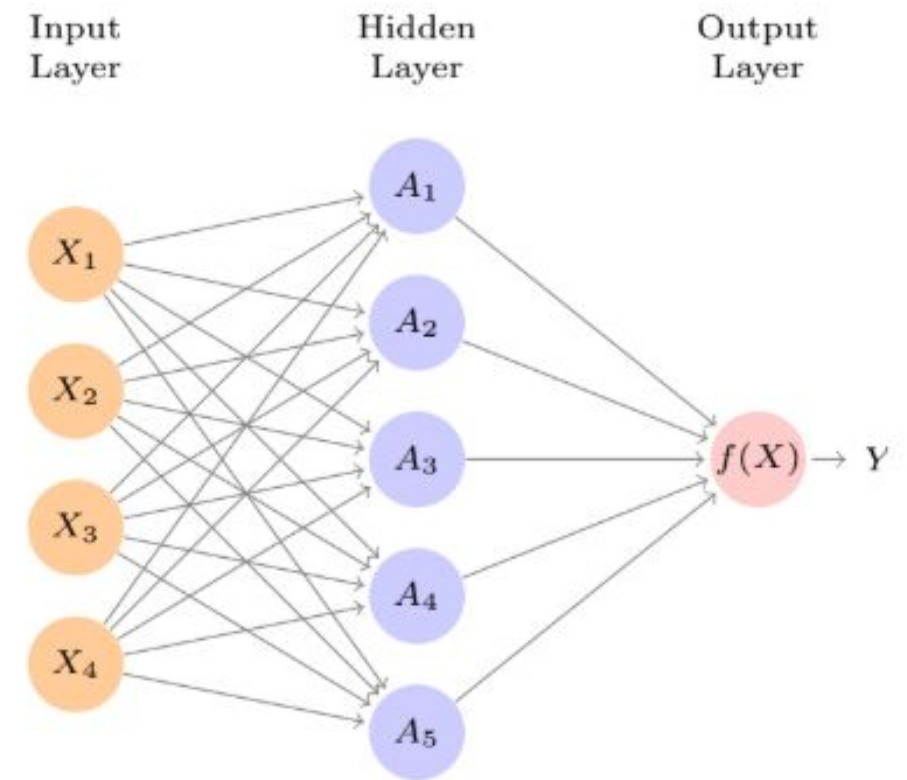
The idea behind neural networks, is that by creating **brain-inspired software**, we might be able to get better performance for difficult-to-reason-about problems that the human mind is so good at solving.

Although neural networks are **inspired** by the brain, they are not at all identical.

- It's like the difference between a bird and an airplane.
- They achieve the same end goal using roughly the same strategies, but with major differences in their details.

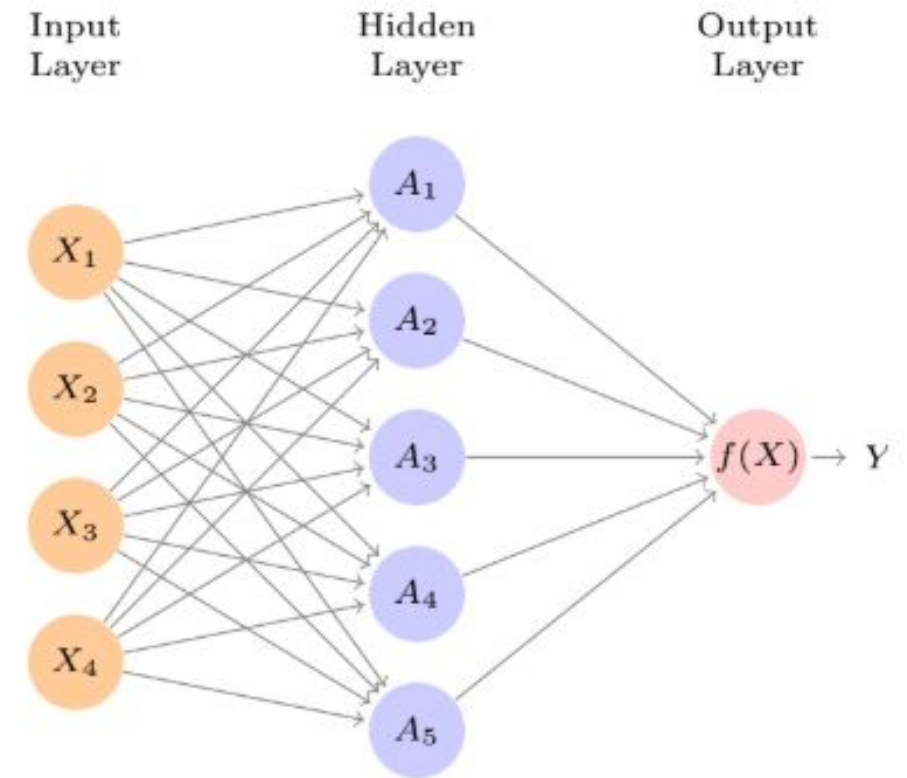
Neural Networks (NNs)

- A **Neural Network** is a kind of machine learning model that is inspired by the structure and functions of **biological neural networks**.
- It consists of **nodes (*neurons*)** connected by **edges (*synapses*)**.



Neural Networks (NNs)

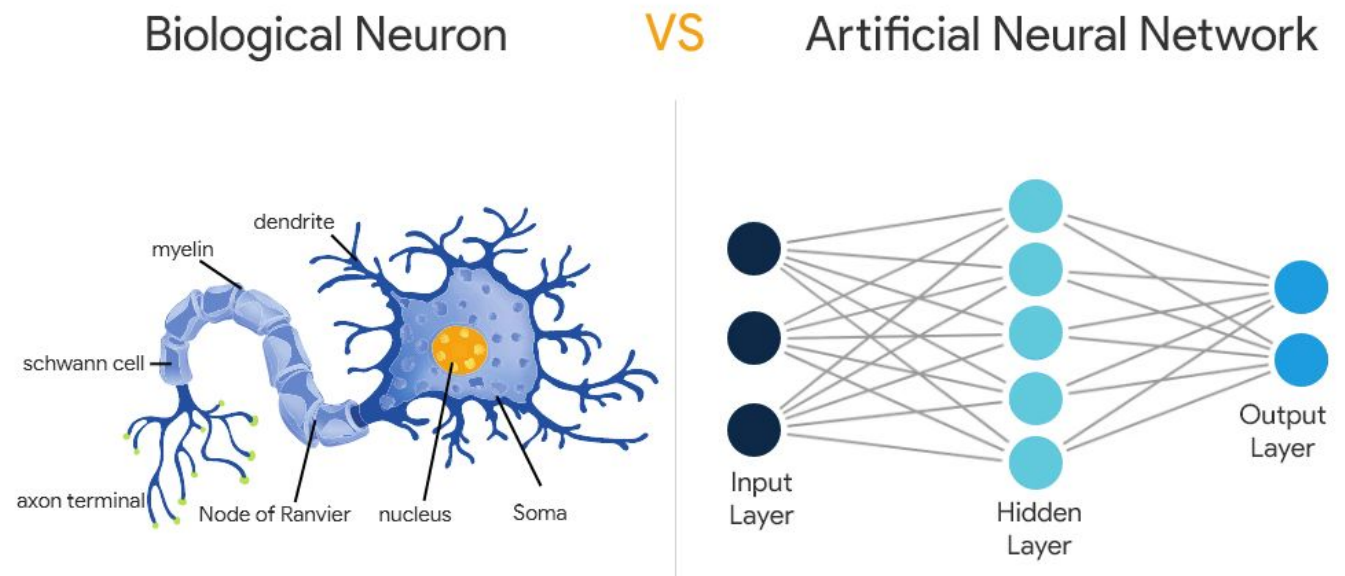
- Each **node (neuron)** receives **signals** from connected neurons, then processes them and sends a signal to other connected neurons.
- The output of each neuron is computed by some non-linear function, called the **activation function**.
- The **strength** of the signal at each connection is determined by a **weight**, which adjusts during the learning process.



Neural Networks: Biological Inspiration

Neural Networks are inspired by the brain.

- A "**neuron**" in a NN can be thought of as a **function** that takes in data from the previous layers as input and produces some output.
 - The number inside the neuron is called its "activation".
 - When a neuron is activated, it "fires".
- This can cause other neurons to fire, since there are connections between neurons.



Neural Networks (NNs)

- Typically, neurons are organized into **layers**. Different layers may perform different transformations on their inputs.
- Our hope is that each layer learns to recognize some element of what we wish to predict.
 - E.g. for image recognition: one layer learns to recognize edges, another learns to recognize curves, etc.

Neural Networks are Powerful

Several studies suggest that even a **single-hidden-layer network**, given enough nodes, and the right set of weights, we **can model any function**.

- Actually learning that function is the hard part.
- You might think of your neural network as being a bit like the C programming language. The language, like any other modern language, is capable of expressing any computable program. But actually coming up with a program that meets your specifications is the hard part.

How it Works

A neural network is composed of **layers of neurons**:

- **Input layer:** takes in the features (e.g., pixels, words)
- **Hidden layers:** transform the data through weighted connections and non-linear functions.
- **Output layer:** produces the final prediction (e.g., class label, probability, regression value).

How it Works

Each **neuron** performs a simple mathematical operation:

$$\hat{y} = \text{activation}(W \cdot x + b)$$

W - weights (learned during training)

x - inputs from previous layer

b - bias term

The **activation function** introduces non-linearity, we'll discuss it in a few slides.

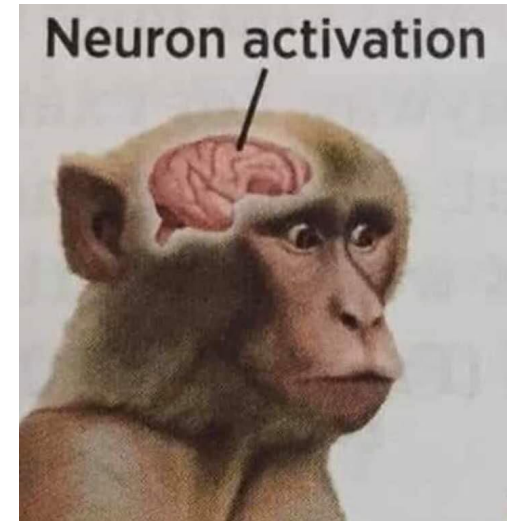
Neural Networks (NNs)

- Each **neuron** has its own set of **weights** (along inbound edges) and its own **bias**.
 - The weights tell you how strongly it's activated.
 - Bias tells you how much the weighted sum has to be before the neuron is meaningfully activated.
- At each layer of the NN, you take the weighted sum of activations at a given layer.

Weights

$$\hat{y} = \text{activation}(W \cdot x + b)$$

$$W = [w_1, w_2, \dots, w_n]$$



Each input to a neuron is multiplied by a weight that determines **how important** each input is to the neuron.

- Large positive weight $\rightarrow$ this input strongly increases the neuron's activation.
- Large negative weight $\rightarrow$ this input strongly decreases the neuron's activation.
- Small or zero weights mean $\rightarrow$ this input has little to no influence on the neuron's activation.

Bias

The bias (b) the weighted sum up or down. It decides how much input is needed for the neuron to activate.

- Large positive bias $\rightarrow$ neuron activates even with weak inputs.
- Large negative bias $\rightarrow$ neuron needs strong evidence to activate.

Activation Functions

Activation functions introduce **non-linearities**. Without activation functions, neural networks would be **restricted to modeling only linear relationships** between inputs and outputs.

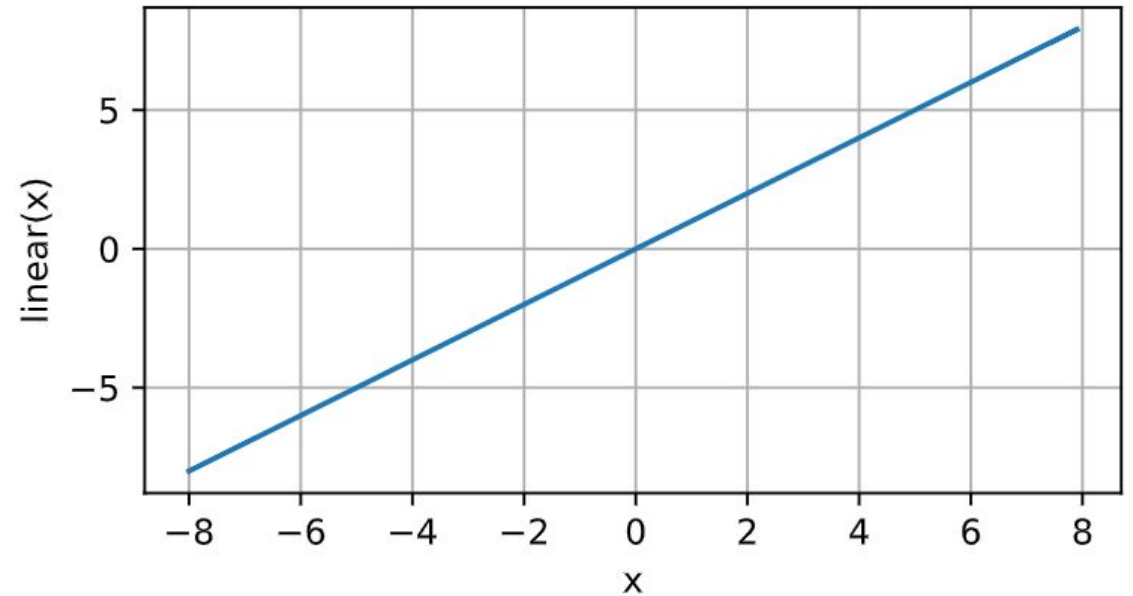
The activation function decides if the neuron should:

- **Activate** → pass information to the next layer.
- **Stay Inactive** → do not pass information to the next layer.

Activation Functions: Linear

The **Linear activation** simply outputs the input value unchanged.

$$\text{linear}(x) = x$$

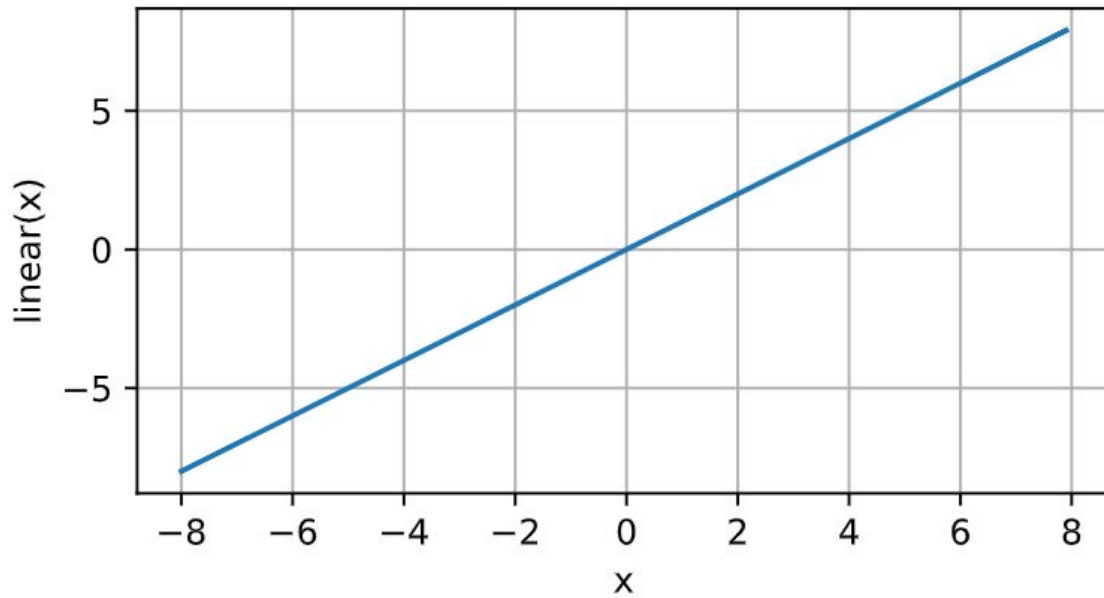


While it avoids vanishing gradients, it **cannot introduce nonlinearity**, so stacking linear layers is equivalent to one linear transformation.

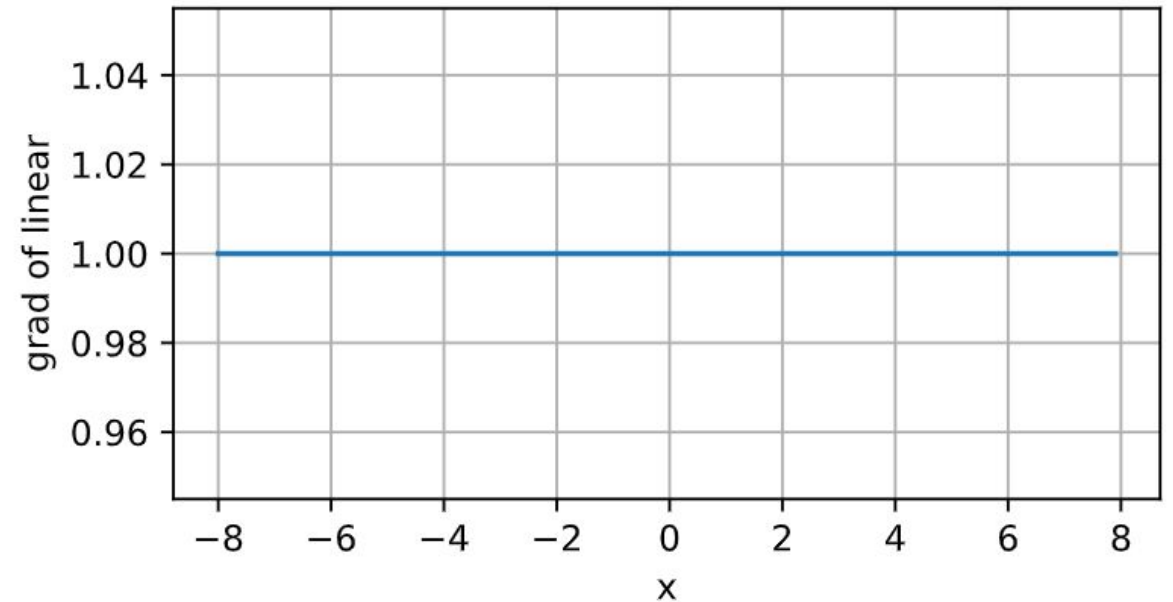
Linear

$$\text{linear}(x) = x$$

Linear Function



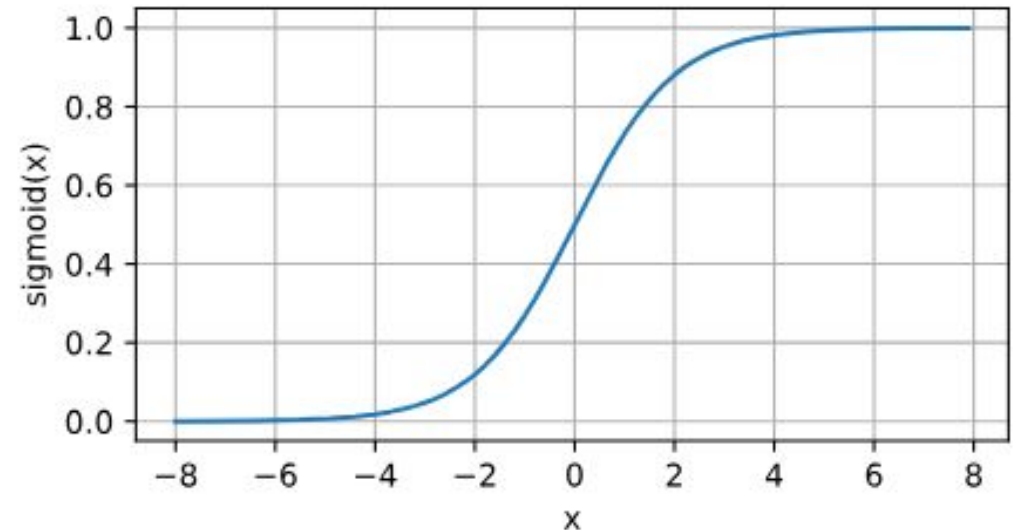
Derivative of the Linear Function



Activation Functions: Sigmoid

The **sigmoid function** transforms inputs in the range $(-\infty, \infty)$ to some value in the range $(0, 1)$.

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

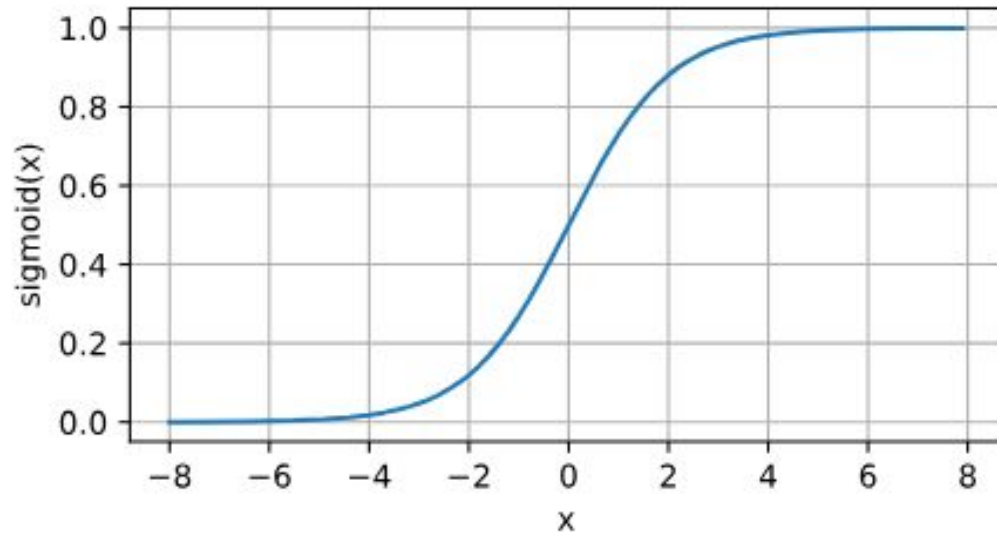


The sigmoid poses challenges for optimization since its **gradient vanishes** for large positive and negative arguments.

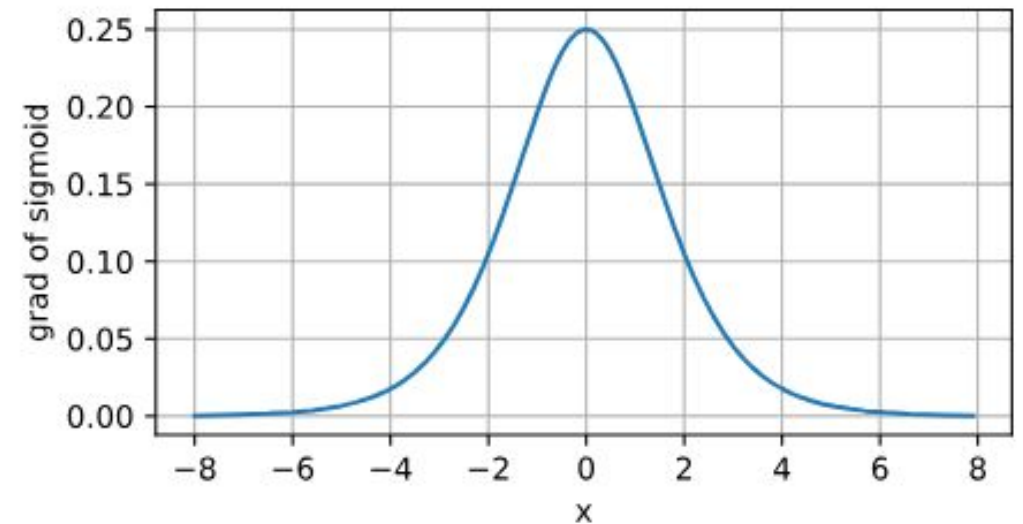
Sigmoid

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid Function



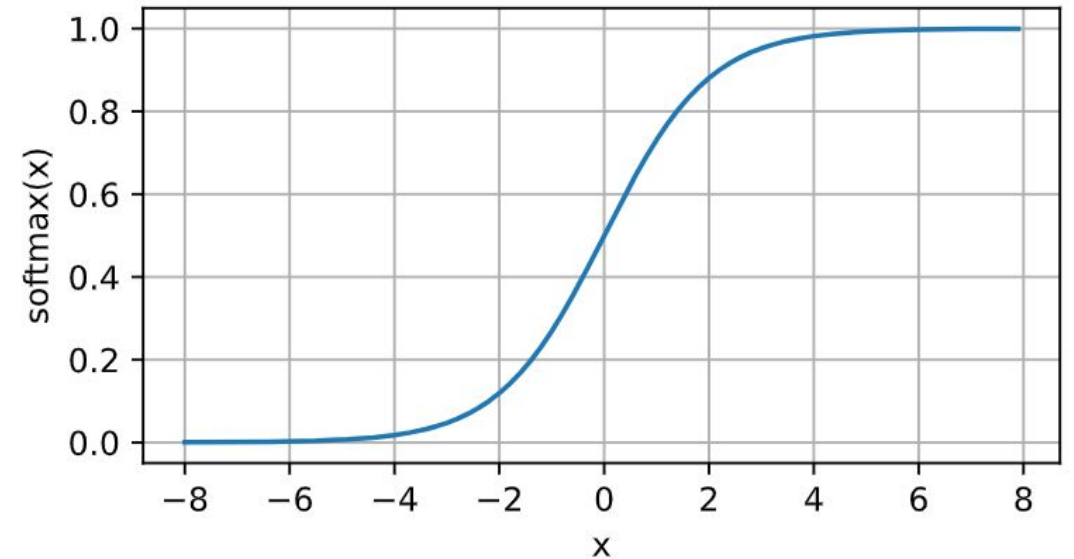
Derivative of the Sigmoid Function



Activation Functions: Softmax

The **Softmax function** converts inputs into probabilities that add up to one.

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

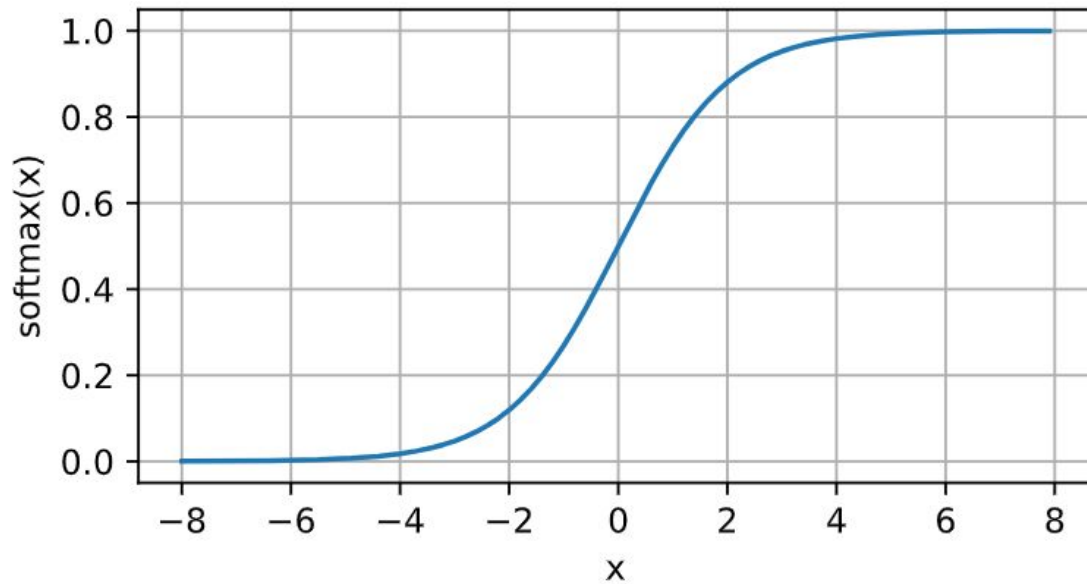


Like the sigmoid, Softmax can suffer from vanishing gradients.

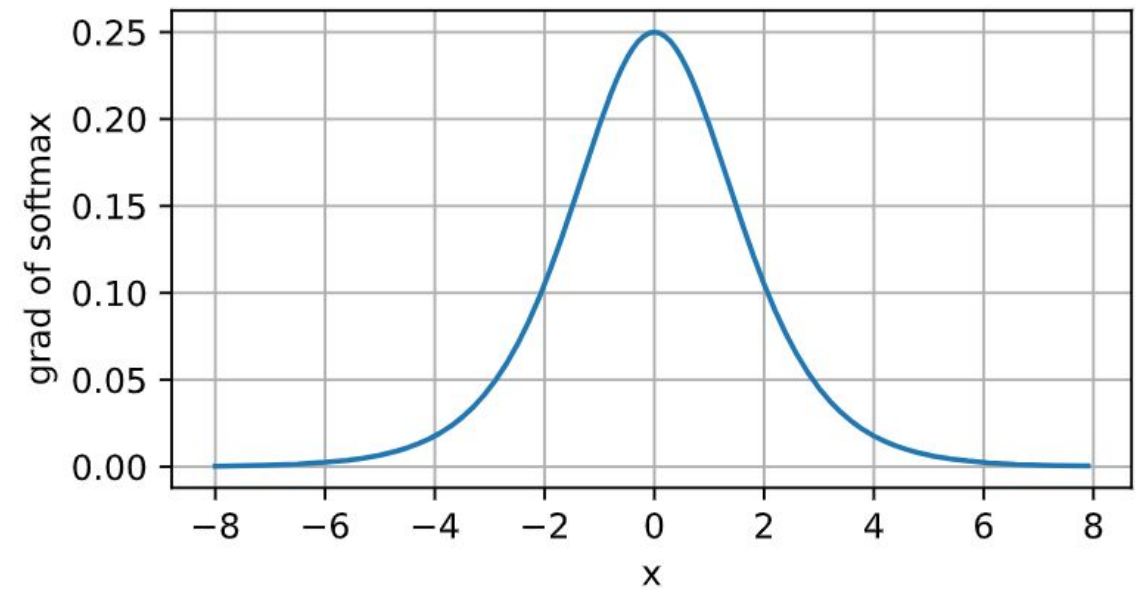
Softmax

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

Softmax Function



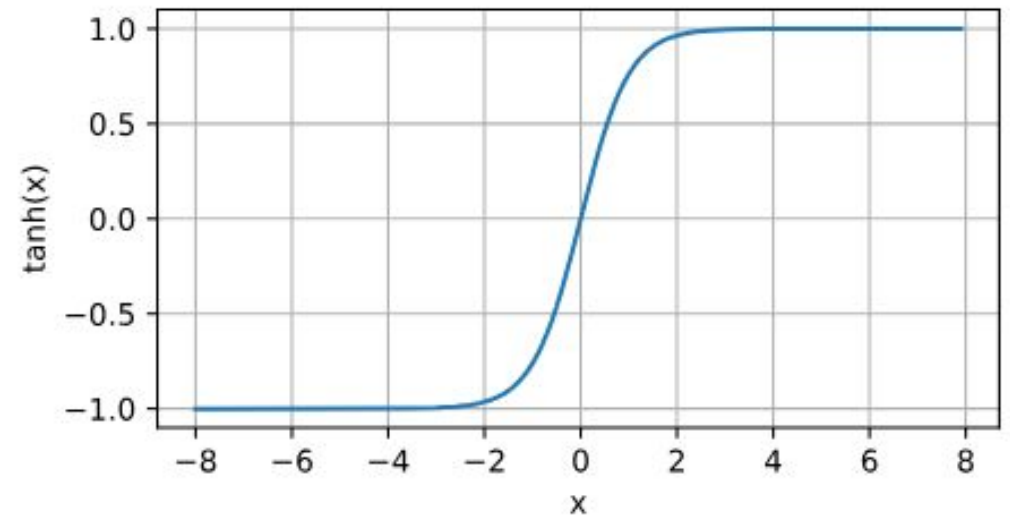
Derivative of the Softmax Function



Activation Functions: Tanh

The **tanh (hyperbolic tangent)** function is similar to the sigmoid function, except it constrains inputs on the interval between -1 and 1.

$$\tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

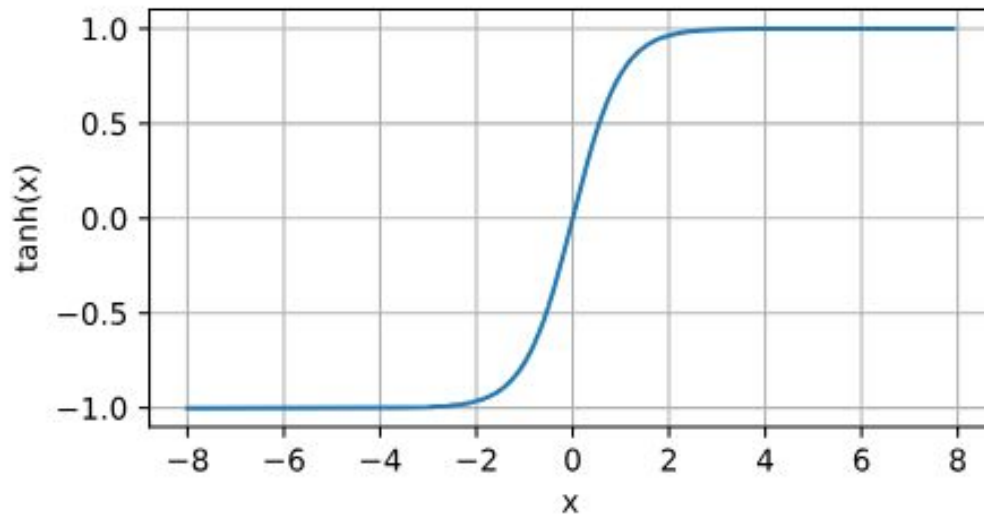


Tanh is also susceptible to the vanishing gradient problem.

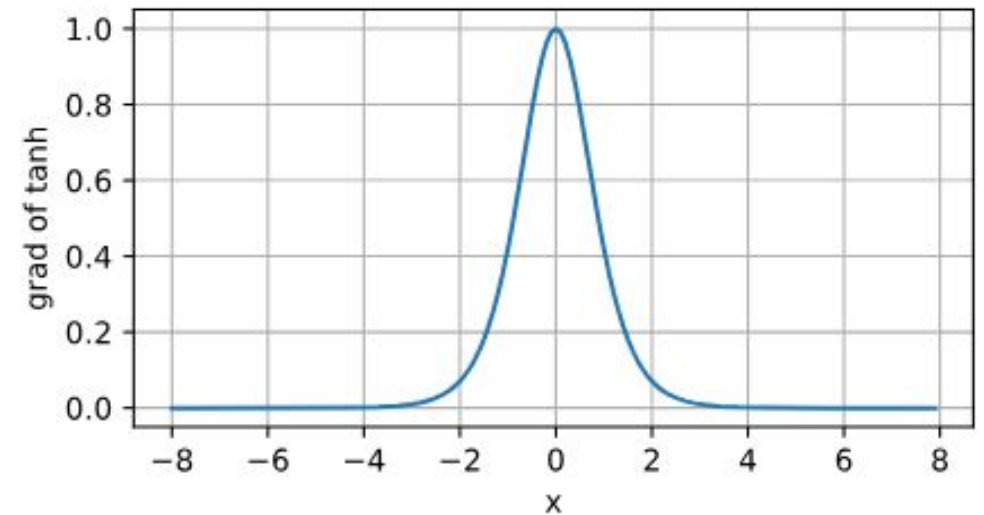
Tanh

$$\tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

Tanh Function



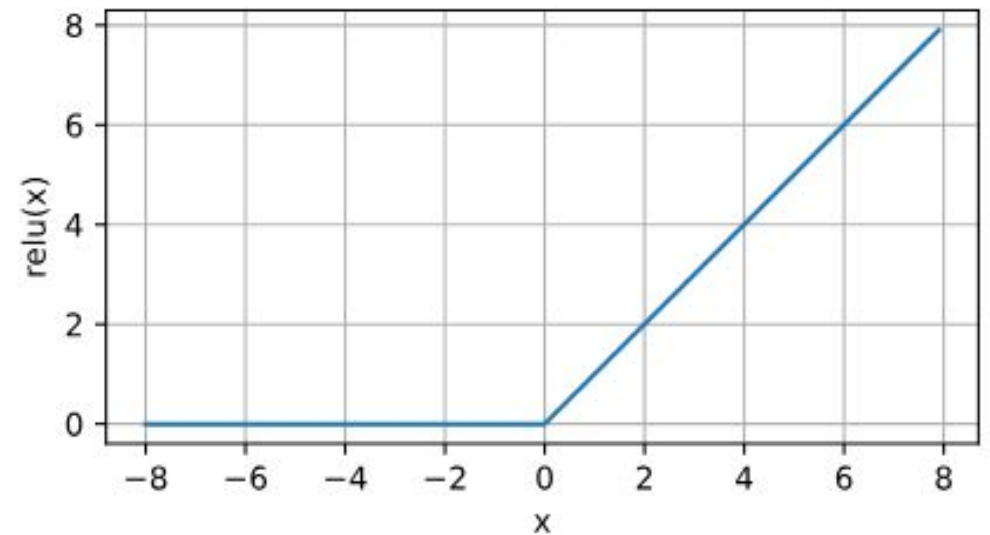
Derivative of the tanh Function



Activation Functions: ReLU

ReLU (rectified linear unit) is one of the most popular activation functions since it is simple to implement and has good performance on a variety of predictive tasks.

$$\text{ReLU}(x) = \max(x, 0)$$

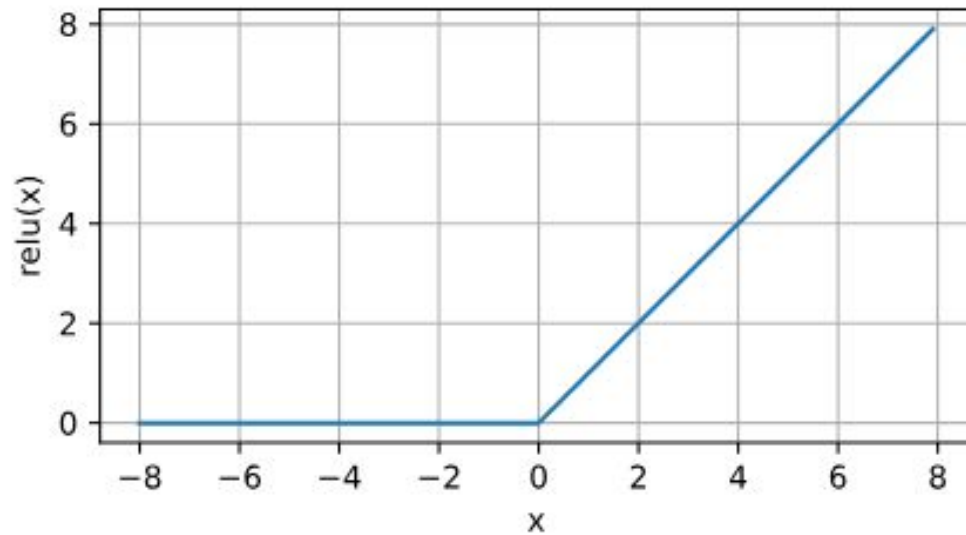


When the **input is negative**, the derivative of the ReLU function is 0, and when the **input is positive**, the derivative of the ReLU function is 1.

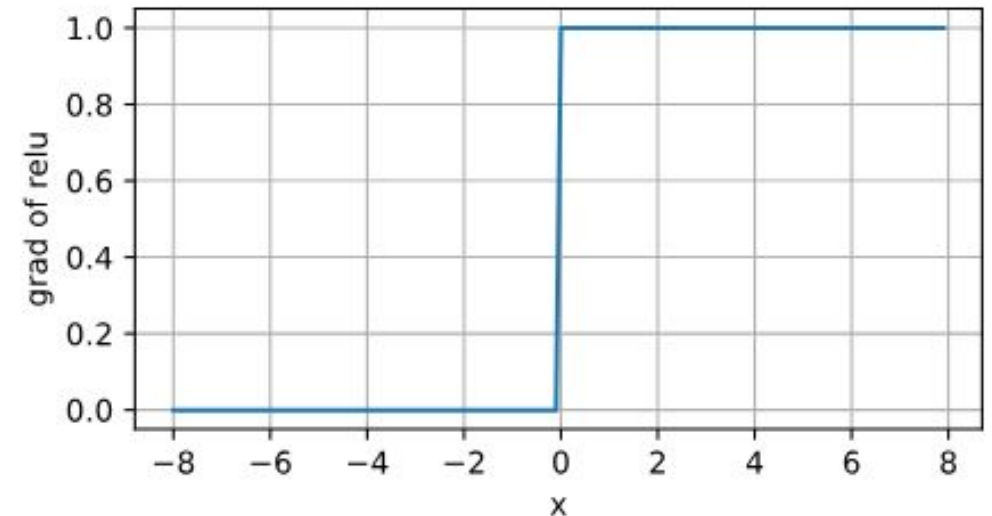
ReLU

$$\text{ReLU}(x) = \max(x, 0)$$

ReLU Function



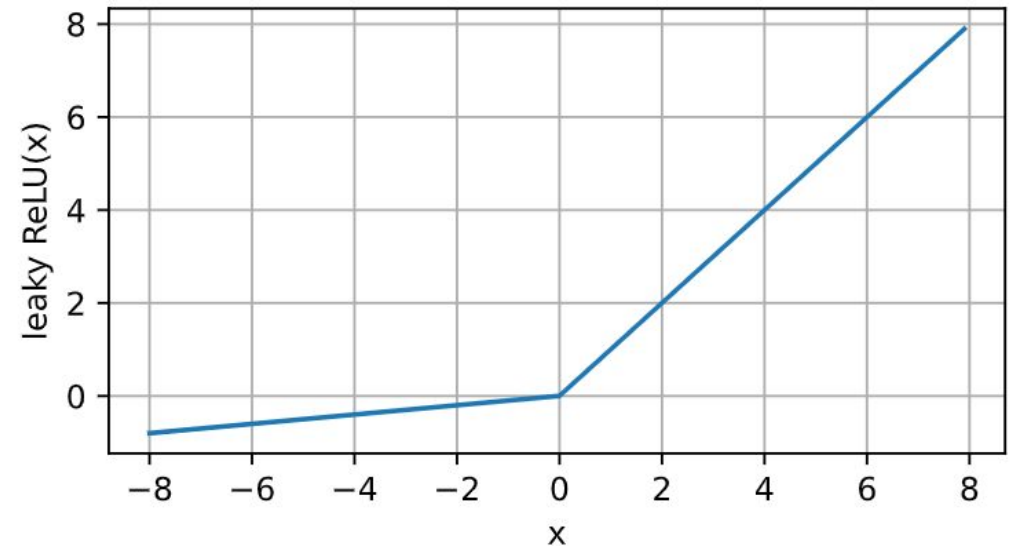
Derivative of the ReLU Function



Activation Functions: Leaky ReLU

The **Leaky ReLU** activation outputs the input for positive values and a scaled version (e.g., $0.01x$) for negative values.

$$\text{leaky ReLU}(x) = \max(0.01x, x)$$

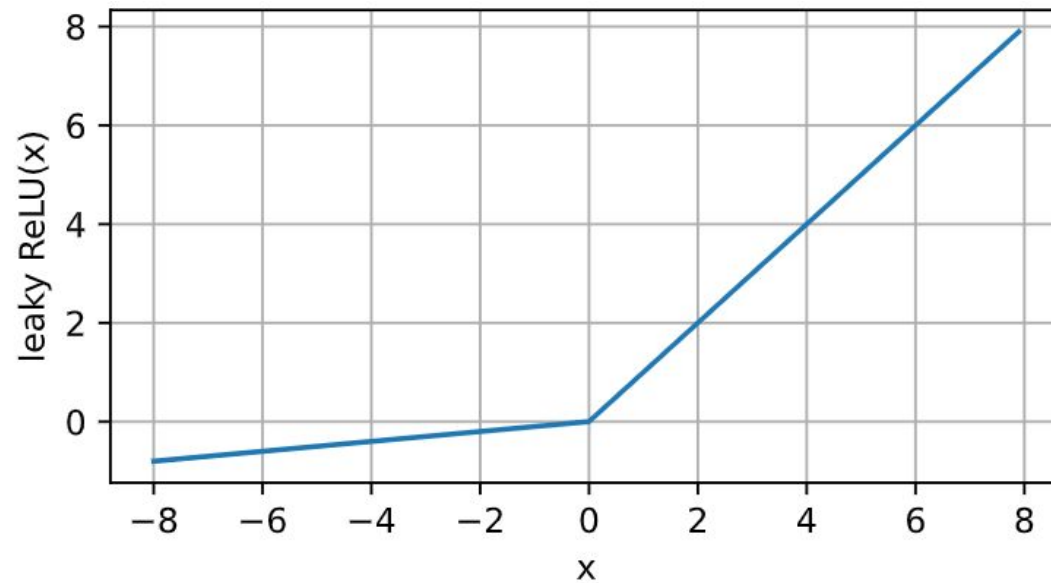


This prevents neurons from dying by maintaining a small gradient even when the input is negative.

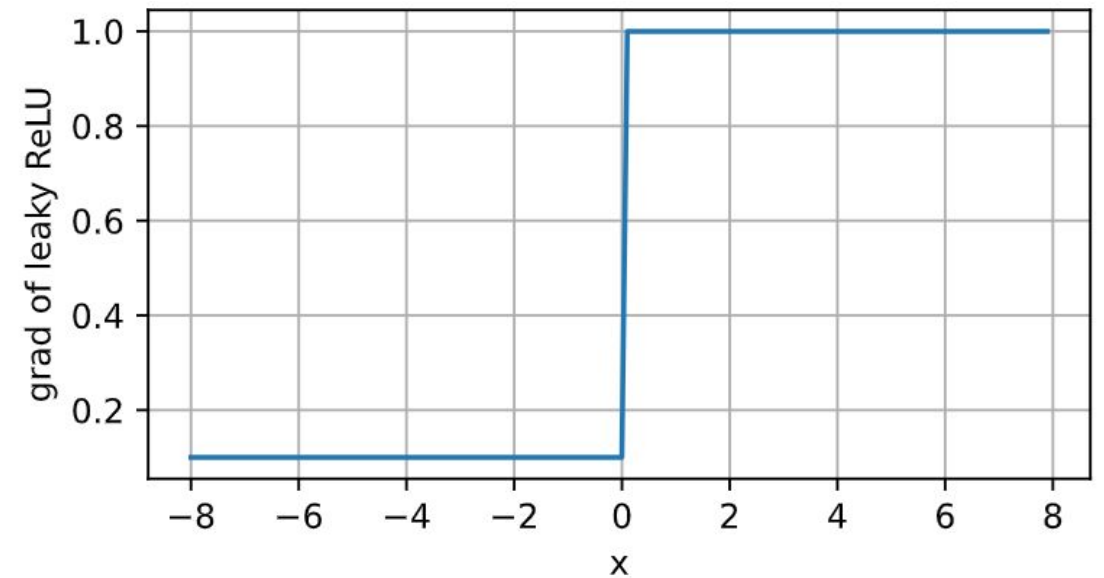
Leaky ReLU

$$\text{leaky ReLU}(x) = \max(0.01x, x)$$

Leaky ReLU Function



Derivative of Leaky ReLU



Many More Activation Functions!

<code>nn.ELU</code>	<code>nn.PReLU</code>	<code>nn.SiLU</code>
<code>nn.Hardshrink</code>	<code>nn.ReLU</code>	<code>nn.Mish</code>
<code>nn.Hardsigmoid</code>	<code>nn.ReLU6</code>	<code>nn.Softplus</code>
<code>nn.Hardtanh</code>	<code>nn.RReLU</code>	<code>nn.Softshrink</code>
<code>nn.Hardswish</code>	<code>nn.SELU</code>	<code>nn.Softsign</code>
<code>nn.LeakyReLU</code>	<code>nn.CELU</code>	<code>nn.Tanh</code>
<code>nn.LogSigmoid</code>	<code>nn.GELU</code>	<code>nn.Tanhshrink</code>
<code>nn.MultiheadAttention</code>	<code>nn.Sigmoid</code>	<code>nn.Threshold</code>
		<code>nn.GLU</code>

There are [many other activation functions](#) beyond the ones covered in this lecture, each with different properties that affect how a network learns and represents complex patterns.

Choosing an Activation Function

- All **hidden layers** usually use the **same activation function**.
- The **output layer** will typically use a **different activation function** from the hidden layers. The choice of activation function **should align with the nature** of the problem (e.g., classification vs. regression).
- **ReLU** should **only be used in the hidden layers**.
- Sigmoid and Tanh functions **should not** be used in hidden layers as they make the model more susceptible to problems during training (due to vanishing gradients).

Choosing an Activation Function

- **Linear:** Use for regression problems.
 - The network should be able to predict any real number, not squash it into a specific range.
- **Sigmoid:** Suitable for binary classification problems.
- **Softmax:** Ideal for multi-class classification problems.
- **ReLU:** Used in Hidden layers of many NNs.
- **Leaky ReLU:** Use if you encounter dying neurons.

Choosing an Activation Function

- **Linear:** Use for regression problems.
- **Sigmoid:** Suitable for binary classification problems.
 - Produces smooth probabilities for “yes/no” outcomes.
- **Softmax:** Ideal for multi-class classification problems.
- **ReLU:** Used in Hidden layers of many NNs.
- **Leaky ReLU:** Use if you encounter dying neurons.

Choosing an Activation Function

- **Linear:** Use for regression problems.
- **Sigmoid:** Suitable for binary classification problems.
- **Softmax:** Ideal for multi-class classification problems.
 - Assigns a probability to each target class
- **ReLU:** Used in Hidden layers of many NNs.
- **Leaky ReLU:** Use if you encounter dying neurons.

Choosing an Activation Function

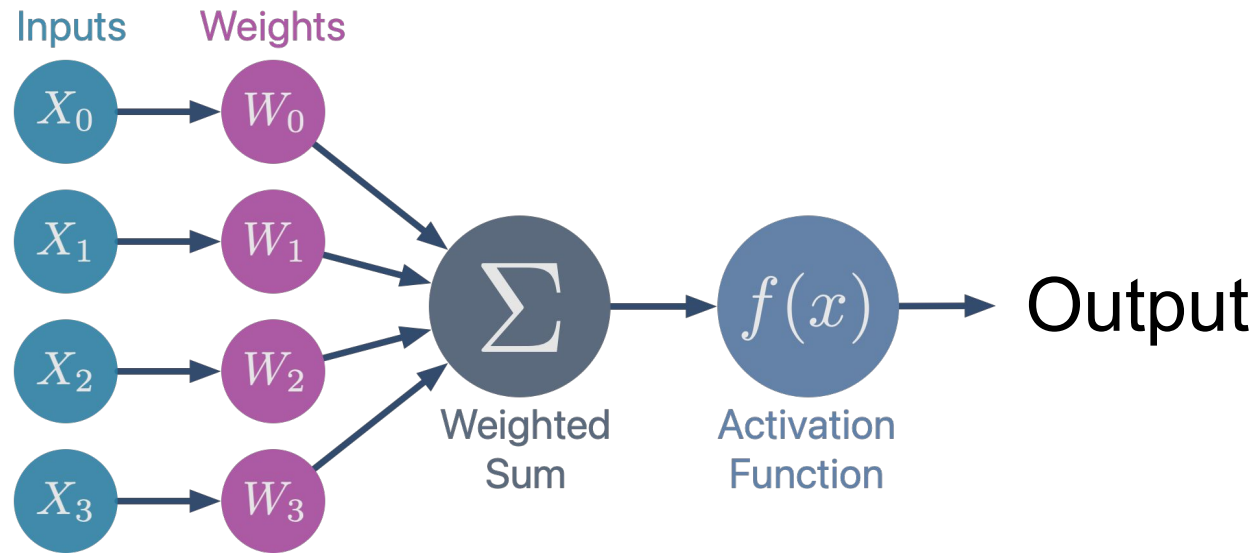
- **Linear:** Use for regression problems.
- **Sigmoid:** Suitable for binary classification problems.
- **Softmax:** Ideal for multi-class classification problems.
- **ReLU:** Used in Hidden layers of many NNs.
 - Keeps gradients flowing for positive activations, sparse for negative
- **Leaky ReLU:** Use if you encounter dying neurons.

Choosing an Activation Function

- **Linear:** Use for regression problems.
- **Sigmoid:** Suitable for binary classification problems.
- **Softmax:** Ideal for multi-class classification problems.
- **ReLU:** Used in Hidden layers of many NNs.
- **Leaky ReLU:** Use if you encounter dying neurons.
 - Prevents neurons from going completely silent. A small negative slope ensures continuous gradient flow.

Perceptrons

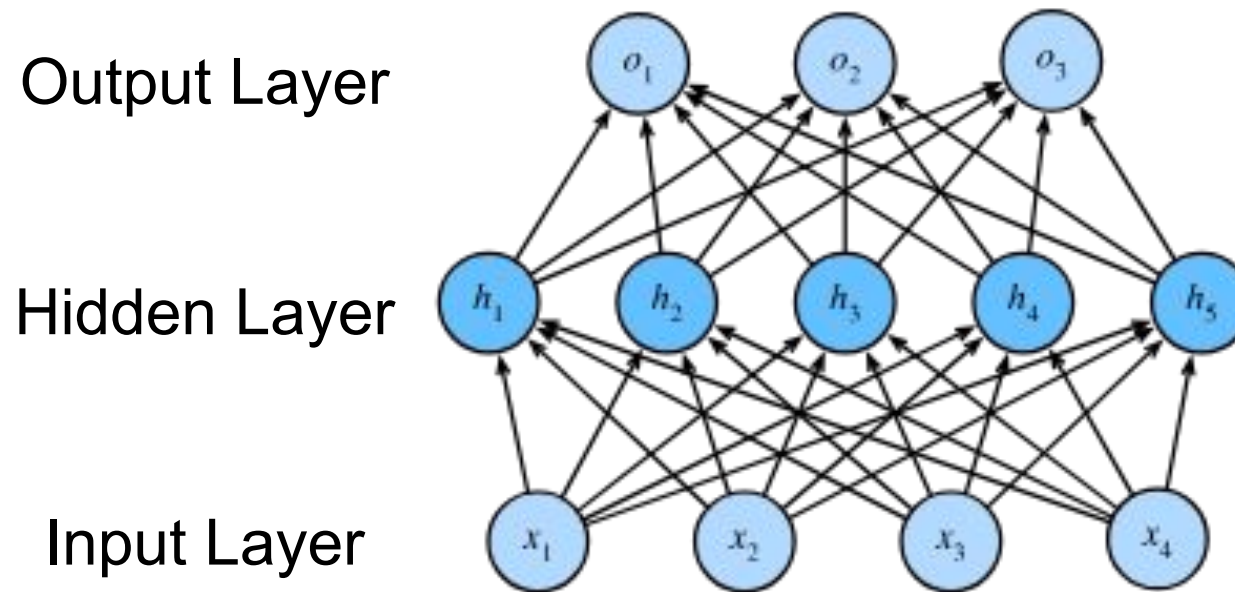
A Perceptron is the simplest possible Neural Network architecture. It is essentially a single "neuron".



They are the building blocks of more complicated networks.

Multilayer Perceptrons (MLPs)

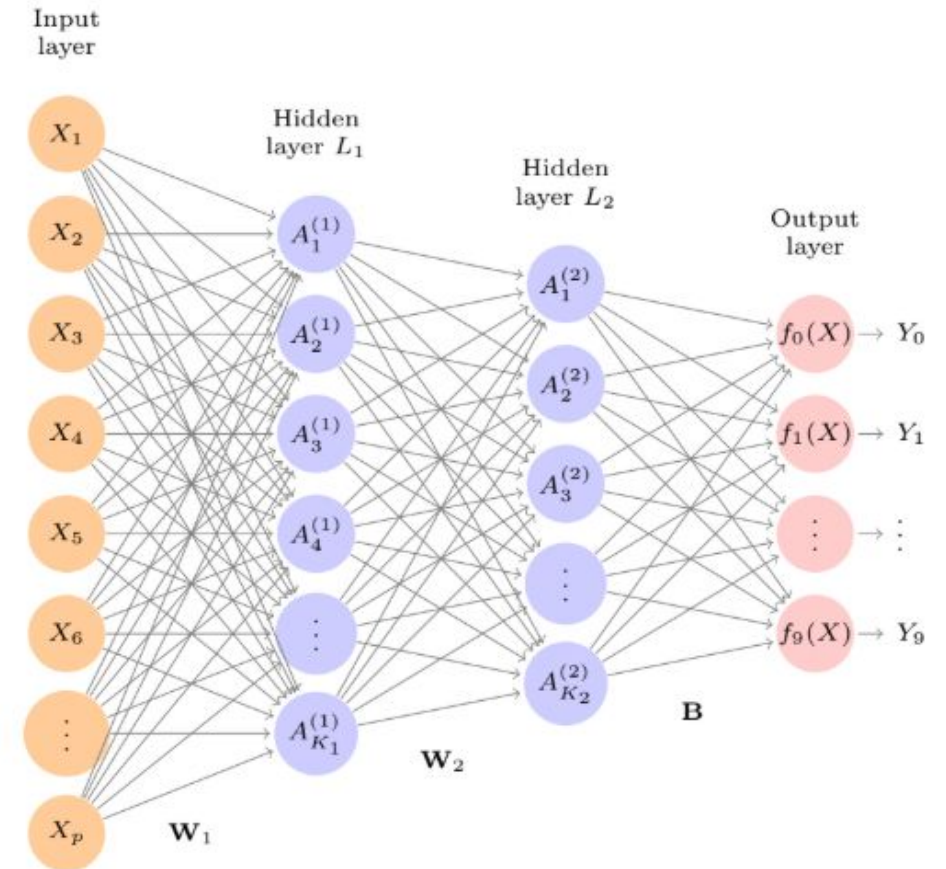
Multilayer perceptrons (MLPs) consist of multiple layers of neurons each fully connected to those in the layer below (from which they receive input) and those above (which they influence).



More Layers

Most of the time, neural networks typically have **more than one hidden layer**, and often many units per layer.

- A NN with many hidden layers is **deep**.
- A NN with many units per layer is **wide**.



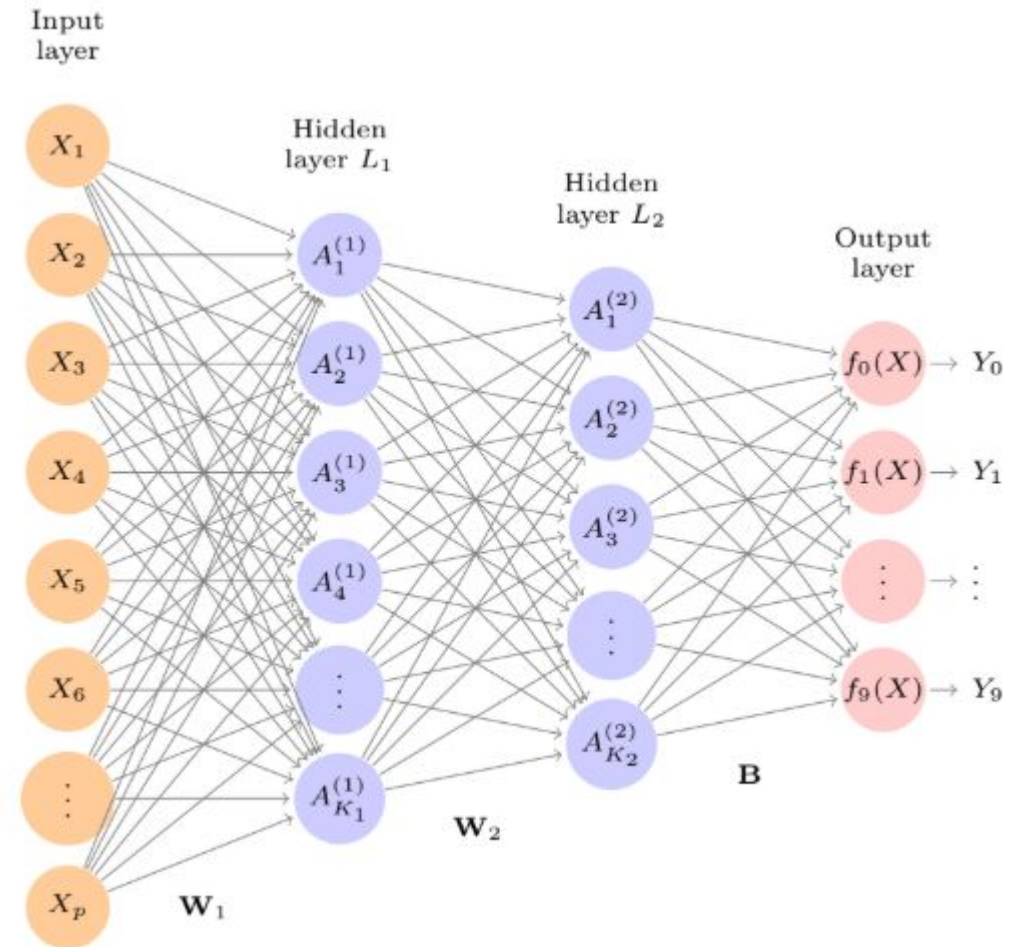
More Layers

The input and output layers will be decided by the problem. You still have to decide:

1. How many **hidden layers** you want.
2. How many **units (neurons)** should be in each hidden layer.
 - Having many neurons can allow for more complexity, but can risk **overfitting**.

Regularization

The diagram here shows a NN architecture for a digit recognition task. Even with just 2 layers, this network could easily have **hundreds of thousands** of parameters to train! Since the number of parameters in a neural network can get large very quickly, we need **regularization methods** to help **avoid overfitting**.



Regularization

There are several regularization techniques for NNs:

- **L1 and L2 regularization** - apply to the loss function.
 - Penalize large weights encouraging the network to keep weights small and distributed, but rarely exactly zero.
- **Early stopping** - stopping the training of the neural network at an earlier epoch.
- **Dropout** - randomly ignore a fraction of neurons during training, forcing the network to not rely too heavily on any single neuron.

Training Neural Networks

- Training a neural network ("**learning**") refers to finding values for all of the neural network's **weights and biases** that minimize a certain **cost function**.
- A **cost function** is a measure of error between what value your model predicts and what the value actually is.
- During the training process, the neural network **adjusts its weights and biases** to **minimize** the cost function. The goal is to find the **minimum value** of the cost function, which corresponds to the **best set of weights and biases** that make accurate predictions.

Cost Functions

There are many different cost functions, and the choice of cost function depends on the problem being solved.

- **MSE** is used for regression problems.
- **Binary Cross-Entropy** is used for binary classification.
- **Categorical Cross-Entropy** is used for multi-class classification problems.

Broadly speaking, **Cost functions** are a measure of how well our model performs. **Activation functions** transform the value at a given neuron in an attempt to improve our model's performance.

How do we Train NNs?

1. **Initialize** all weights and biases at random.
2. **Forward pass** - Pass the training data through the network layer by layer to produce an output (predictions).
3. **Compute the loss** using a **cost function**.
4. **Backward pass (Backpropagation)** - Compute how much each weight contributed to the loss.
5. **Optimization** - Update the weights.
6. Repeat steps 2-5 many times.
7. **Evaluate** on test data.

How do we Train NNs?

```
# Reset all stored gradients from the previous iteration
optimizer.zero_grad()

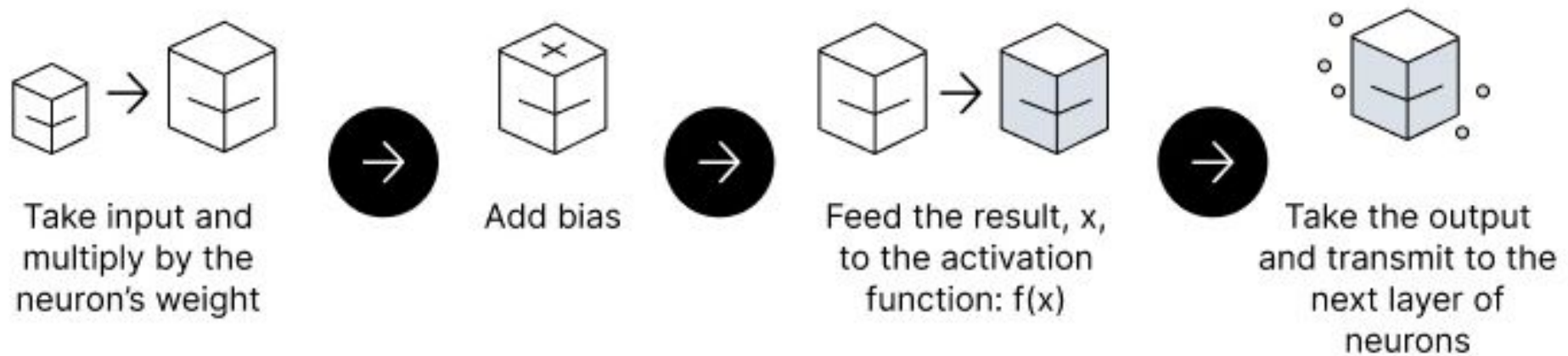
# Perform a forward pass: feed the input batch through the model to get predictions
outputs = model(inputs)

# Compute the loss between the model's predictions and the true labels
loss = criterion(outputs, targets)

# Perform backpropagation: compute gradients of the loss with respect to model parameters
loss.backward()

# Update the model parameters using the optimizer (gradient descent step)
optimizer.step()
```


Forward Pass



Backpropagation

Backpropagation is an algorithm used to compute the gradient of the cost function.

Mathematically, the goal of backpropagation is to compute the partial derivatives $\partial C / \partial w$ and $\partial C / \partial b$ of the cost function C with respect to any weight w or bias b in the network.

Gradient Descent (GD)

Gradient Descent (GD) works by iteratively adjusting the model's weights and biases to minimize the loss function (the measure of prediction error).

At each step, it **computes the gradient (slope) of the loss** with respect to each parameter and updates them in the **opposite direction** of the gradient (moving "downhill" toward lower error).

Stochastic Gradient Descent (SGD)

Stochastic gradient descent (SGD) is basically gradient descent, but instead of relying on the entire dataset to compute the gradient at each step, SGD uses a single data sample at a time.

Mini-Batch Gradient Descent

It is **computationally intensive** to compute the gradient descent steps for all training samples, so we can use **mini-batches** instead.

- Shuffle the training data and randomly divide them into equally-sized subsets.
- Calculate the gradient for each subset.
- This gives a good approximation to what's the best "step" to take.

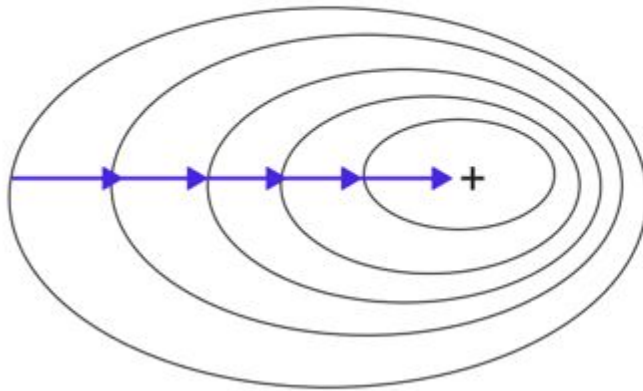
This method still converges to a **local minimum** of the cost function, but takes less computation and less time than regular GD.

Gradient Descent Comparison

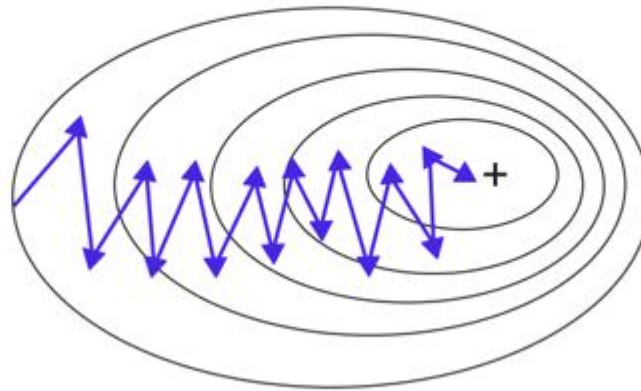
Gradient Descent	Stochastic Gradient Descent	Mini-Batch Gradient Descent
• Uses all training data • Slow • Very stable, smooth updates	• Updates using one sample at a time • Fast • Updates jump around	• Uses a small batch of samples for updates • Fast • Updates more noisy than the GD, but smoother than SGD

Gradient Descent Comparison

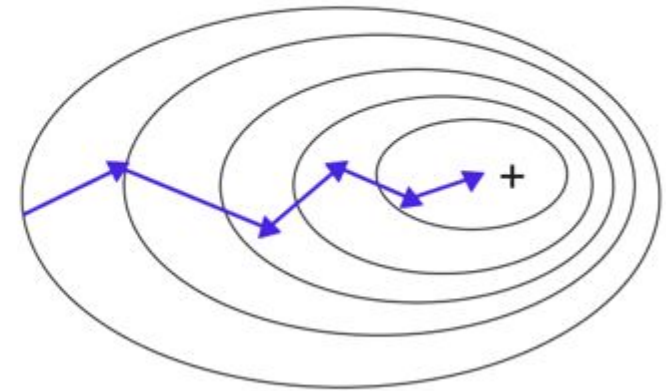
Batch Gradient Descent



Stochastic Gradient Descent



Mini-Batch Gradient Descent



Other NN Architectures

- **Convolutional neural networks (CNNs)** are used for image recognition and computer vision tasks, such as classification and object detection.
- **Recurrent neural networks (RNNs)** are used for time series and other sequences where past information influences future predictions.
- **Long short-term memory networks (LSTMs)** can remember long-term dependencies, often used in speech recognition and language modeling.
- **Residual Networks (ResNets)** work well for sequence data and used for sentiment analysis, spam filters, etc.
- **Transformers** originally used for natural language processing and other sequence tasks (e.g. GPTs), now they can do everything.

Challenges with NNs

- **Vanishing Gradients:** Gradients in early layers become extremely small, causing slow or stalled learning.
→ *ReLU helps with this.*
- **Exploding Gradients:** Gradients grow uncontrollably large, leading to unstable training.
→ *Use batch normalization or try lowering the learning rate.*
- **Dead ReLU Units:** Some ReLU neurons output zero for all inputs and stop learning.
→ *Lower the learning rate or use Leaky ReLU to keep neurons active.*

Summary

- **Neural networks** are a class of very powerful and flexible machine learning models inspired by biological neural networks.
- They require **activation functions** to introduce non-linearity into them (e.g. ReLU, Sigmoid, and many others).
- **Regularization** is important because they can become overly complex very quickly.
- **Backprop and gradient descent** are used to train NNs - they help us find the optimal values for the **weights and biases**.
- There are lots of challenges in training NNs, and lots of applications of them in the real-world.

Further Resources

- [3Blue1Brown: Neural Networks, Conceptually](#)
- [3Blue1Brown: Backprop Intuitively](#)
- [Activation Functions](#)
- [Google Backprop](#)
 - [Google - High Level Video on BackProp](#)